Question 1

> Show that the set S defined by $1 \in S$ and $s + t \in S$ whenever $s \in S$ and $t \in S$ is the set of positive integers.

Let S be the set defined by $1 \in S$ and $s+t \in S$ whenever $s \in S$ and $t \in S$. We will show that S is the set of positive integers by induction.

Basis Step: $1 \in S$.

Inductive Step: Suppose that $1, 2, 3, \dots, n \in S$ for some $n \in \mathbb{N}$. We will show that $n + 1 \in S$. Since $1 \in S$, we have $1 + n = n + 1 \in S$. Therefore, $1, 2, 3, \dots, n, n + 1 \in S$.

By the principle of mathematical induction, we conclude that S is the set of positive integers $\mathbb{N}$. ■

Question 2

Consider the recursively defined function with two parameters m and n. The basis step is $a_{1,1} = 5$.

The recursive step is $a(m,n) = \begin{cases} a_{m-1,n} + 2 & \text{if } n = 1 \text{ and } m > 1 \\ a_{m,n-1} + 2 & \text{if } n > 1 \end{cases}$.

Show that $a_{m,n} = 2(m+n)+1$ when m and n are positive integers.

Let $P(m,n)$ be the statement that $a_{m,n} = 2(m+n)+1$ for positive integers m and n. We will show that $P(m,n)$ is true for all positive integers m and n by induction.

Basis Step: $P(1,1)$ is true since $a_{1,1} = 5 = 2(1+1)+1$.

Inductive Step: Suppose that $P(m,n)$ is true for some positive integers m and n. We will show that $P(m+1,n)$ and $P(m,n+1)$ are true.

1. $P(m+1,n)$: We have $a_{m+1,n} = a_{m,n}+2 = 2(m+n)+1+2 = 2(m+1+n)+1$. Therefore, $P(m+1,n)$ is true.
2. $P(m,n+1)$: We have $a_{m,n+1} = a_{m,n}+2 = 2(m+n)+1+2 = 2(m+n+1)+1$. Therefore, $P(m,n+1)$ is true.

By the principle of mathematical induction, we conclude that $P(m,n)$ is true for all positive integers m and n. ■

Question 3

Ackerman's function:

$$A(m,n) = \begin{cases} 2n & \text{if } m = 0 \\ 0 & \text{if } m \geq 1 \text{ and } n = 0 \\ 2 & \text{if } m \geq 1 \text{ and } n = 1 \\ A(m-1, A(m, n-1)) & \text{if } m \geq 1 \text{ and } n \geq 2 \end{cases}$$

Find these values of Ackerman's function:

a) $A(1,0)$ $A(1,0) = 0.$

b) $A(0,1)$ $A(0,1) = 2n = 2.$

c) $A(1,1)$ $A(1,1) = 2.$

d) $A(2,2)$ $A(2,2) = A(2-1, A(2,2-1)) = A(1, A(2,1)) = A(1,2)$
$= A(1-1, A(1,2-1)) = A(0, A(1,1)) = A(0,2) = 2n = 4.$

Question 4

$$A(m,n) = \begin{cases} 2n & \text{if } m = 0 \\ 0 & \text{if } m \geq 1 \text{ and } n = 0 \\ 2 & \text{if } m \geq 1 \text{ and } n = 1 \\ A(m-1, A(m, n-1)) & \text{if } m \geq 1 \text{ and } n \geq 2 \end{cases}$$

Show that $A(1,n) = 2^n$ whenever $n \geq 1$.

Let $P(n)$ be the statement that $A(1,n) = 2^n$ for $n \geq 1$. We will show that $P(n)$ is true for all $n \geq 1$ by induction.

Basis Step: $P(1)$ is true since $A(1,1) = 2$ and $2^1 = 2$.

Inductive Step: Suppose that $P(k)$ is true for some $k \geq 1$. We will show that $P(k+1)$ is true.

$A(1,k+1) = A(1-1, A(1,k)) = A(0, A(1,k)) = A(0, 2^k) = 2n = 2^{k+1}$.

By the principle of mathematical induction, we conclude that $\forall n \geq 1, P(n)$. ■

Question 5

> Compute the GCD of 20 and 52 using the brute force method that we discussed in class. Provide all divisors of 20 and 52. Provide the list of common divisors. And finally provide the GCD.

Divisors of 20: $\{20, 10, 5, 4, 2, 1\}$

Divisors of 52: $\{52, 26, 13, 4, 2, 1\}$

Common divisors: $\{1, 2, 4\}$

GCD of 20 and 52 is: 4

Question 6

> Compute the GCD of 98 and 154 using the brute force method that we discussed in class. Provide all divisors of 98 and 154. Provide the list of common divisors. And finally provide the GCD.

Divisors of 98: $\{98, 49, 14, 7, 2, 1\}$

Divisors of 154: $\{154, 77, 22, 14, 11, 7, 2, 1\}$

Common divisors: $\{1, 2, 14, 7\}$

GCD of 98 and 154 is: 14

Question 7

Compute the GCD of 20 and 52 using the Euclidean algorithm. Show each step of the algorithm.

Step	Equation
1	$52 = 20 \cdot 2 + 12$
2	$20 = 12 \cdot 1 + 8$
3	$12 = 8 \cdot 1 + 4$
4	$8 = 4 \cdot 2 + 0$

GCD of 20 and 52 is: 4

Question 8

Compute the GCD of 98 and 154 using the Euclidean algorithm. Show each step of the algorithm.

Step	Equation
1	$154 = 98 \cdot 1 + 56$
2	$98 = 56 \cdot 1 + 42$
3	$56 = 42 \cdot 1 + 14$
4	$42 = 14 \cdot 3 + 0$

GCD of 98 and 154 is: 14

Question 9

Give a recursive algorithm for finding a **mode** of a list of integers. (A **mode** is an element in the list that occurs at least as often as every other element.) You can assume that the list contains at least one integer.

1. Base case: If the index reaches the end of the list, return the current mode.
2. If current matches mode, increment the count and go to the next element.
3. If current is different from mode and count > 0, decrement the count and go to the next element.
4. If current is different from the mode and count $= 0$, set the mode to the current element and reset count to 1.

Let L be the list of integers and $P(L, i, \text{count}, \text{mode})$ be the recursive function to find the mode of the list.

$$P(L, i, \text{count}, \text{mode}) = \begin{cases} \text{mode} & \text{if } i = \text{len}(L) \\ P(L, i+1, \text{count}+1, \text{mode}) & \text{if } L[i] = \text{mode} \\ P(L, i+1, \text{count}-1, \text{mode}) & \text{if count} > 0 \\ P(L, i+1, 1, L[i]) & \text{if count} = 0 \end{cases}$$

```rust
fn mode(lst: &[i32]) -> i32 {
    fn mode_reursive(lst: &[i32], i: usize, count: i32, mode: i32) -> i32 {
        if i == lst.len() {
            mode // Base case: reached end of list
        } else if lst[i] == mode {
            mode_reursive(lst, i + 1, count + 1, mode) // Current matches mode
        } else if count > 0 {
            mode_reursive(lst, i + 1, count - 1, mode) // Different, but prev mode had count
        } else {
            mode_reursive(lst, i + 1, 1, lst[i]) // Different, prev mode had count = 0
        }
    }
    mode_reursive(lst, 1, 1, lst[0])
}
```

Question 10

Questions 10 to 12 are about mergesort. For each of the questions, please show how the problems get divided into two parts and how they got merged. For an odd number of elements you can put one more element either in the right part or in the left part but be consistent.

Consider the list $1, 5, 2, 3, 6, 4$. Execute mergesort on this list and show each step of the algorithm.

1. Divide the list into two halves: $L_1 = 1, 5, 2$ and $L_2 = 3, 6, 4$.
2. $L = \text{merge}(\text{mergesort}(L_1), \text{mergesort}(L_2))$.
    1. Divide L_1 into two halves: $L_{11} = 1, 5$ and $L_{12} = 2$.
    2. $L_1 = \text{merge}(\text{mergesort}(L_{11}), \text{mergesort}(L_{12}))$.
        1. $L_{11} = \text{merge}(\text{mergesort}(1), \text{mergesort}(5))$.
            1. $L_{11} = \text{merge}([1], [5]) = [1, 5]$.
        2. $L_{12} = \text{merge}([2], []) = [2]$.
        3. $L_1 = \text{merge}([1, 5], [2]) = [1, 2, 5]$.
    3. Divide L_2 into two halves: $L_{21} = 3, 6$ and $L_{22} = 4$.
    4. $L_2 = \text{merge}(\text{mergesort}(L_{21}), \text{mergesort}(L_{22}))$.
        1. $L_{21} = \text{merge}(\text{mergesort}(3), \text{mergesort}(6))$.
            1. $L_{21} = \text{merge}([3], [6]) = [3, 6]$.
        2. $L_{22} = \text{merge}([4], []) = [4]$.
        3. $L_2 = \text{merge}([3, 6], [4]) = [3, 4, 6]$.
3. $L = \text{merge}([1, 2, 5], [3, 4, 6])$
    1. Add $1 > 2 > 3 > 4 > 5 > 6$.
    2. $L = [1, 2, 3, 4, 5, 6]$.

Question 11

Consider the list 3, 5, 7, 3, 6, 7, 1, 4. Execute mergesort on this list and show each step of the algorithm.

1. Divide the list into two halves: $L_1 = 3, 5, 7, 3$ and $L_2 = 6, 7, 1, 4$.
2. $L = \text{merge}(\text{mergesort}(L_1), \text{mergesort}(L_2))$.
    1. Divide L_1 into two halves: $L_{11} = 3, 5$ and $L_{12} = 7, 3$.
    2. $L_1 = \text{merge}(\text{mergesort}(L_{11}), \text{mergesort}(L_{12}))$.
        1. $L_{11} = \text{merge}(\text{mergesort}(3), \text{mergesort}(5))$.
            1. $L_{11} = \text{merge}([3], [5]) = [3, 5]$.
        2. $L_{12} = \text{merge}(\text{mergesort}(7), \text{mergesort}(3))$.
            1. $L_{12} = \text{merge}([7], [3]) = [3, 7]$.
        3. $L_1 = \text{merge}([3, 5], [3, 7]) = [3, 3, 5, 7]$.
    3. Divide L_2 into two halves: $L_{21} = 6, 7$ and $L_{22} = 1, 4$.
    4. $L_2 = \text{merge}(\text{mergesort}(L_{21}), \text{mergesort}(L_{22}))$.
        1. $L_{21} = \text{merge}(\text{mergesort}(6), \text{mergesort}(7))$.
            1. $L_{21} = \text{merge}([6], [7]) = [6, 7]$.
        2. $L_{22} = \text{merge}(\text{mergesort}(1), \text{mergesort}(4))$.
            1. $L_{22} = \text{merge}([1], [4]) = [1, 4]$.
        3. $L_2 = \text{merge}([6], [7], [1], [4]) = [1, 4, 6, 7]$.
3. $L = \text{merge}([3, 3, 5, 7], [1, 4, 6, 7])$
    1. Add $1 > 3 > 3 > 4 > 5 > 6 > 7 > 7$.
    2. $L = [1, 3, 3, 4, 5, 6, 7, 7]$.

Question 12

Consider the list 8, 17, 6, 2, 16, 5, 13, 8, 11. Execute merge-sort on this list and show each step of the algorithm.

1. Divide the list into two halves: $L_1 = 8, 17, 6, 2, 16$ and $L_2 = 5, 13, 8, 11$.
2. $L = \text{merge}(\text{mergesort}(L_1), \text{mergesort}(L_2))$.
    1. Divide L_1 into two halves: $L_{11} = 8, 17, 6$ and $L_{12} = 2, 16$.
    2. $L_1 = \text{merge}(\text{mergesort}(L_{11}), \text{mergesort}(L_{12}))$.
        1. $L_{11} = \text{merge}(\text{mergesort}(8), \text{mergesort}(17, 6))$.
            1. $L_{11} = \text{merge}([8], [6, 17]) = [6, 8, 17]$.
        2. $L_{12} = \text{merge}(\text{mergesort}(2), \text{mergesort}(16))$.
            1. $L_{12} = \text{merge}([2], [16]) = [2, 16]$.
        3. $L_1 = \text{merge}([6, 8, 17], [2, 16]) = [2, 6, 8, 16, 17]$.
    3. Divide L_2 into two halves: $L_{21} = 5, 13$ and $L_{22} = 8, 11$.
    4. $L_2 = \text{merge}(\text{mergesort}(L_{21}), \text{mergesort}(L_{22}))$.
        1. $L_{21} = \text{merge}(\text{mergesort}(5), \text{mergesort}(13))$.
            1. $L_{21} = \text{merge}([5], [13]) = [5, 13]$.
        2. $L_{22} = \text{merge}(\text{mergesort}(8), \text{mergesort}(11))$.
            1. $L_{22} = \text{merge}([8], [11]) = [8, 11]$
        3. $L_2 = \text{merge}([5, 13], [8, 11]) = [5, 8, 11, 13]$.
3. $L = \text{merge}([2, 6, 8, 16, 17], [5, 8, 11, 13])$
    1. Add $2 > 5 > 6 > 8 > 8 > 11 > 13 > 16 > 17$.
    2. $L = [2, 5, 6, 8, 8, 11, 13, 16, 17]$.

Question 13

Consider the following program:

```
procedure multyply_neg_one(n):
  m := -1
  i := 1
  while i < n:
    i := i + 1
    m := m.(-1)
```

Show that after executing the program for a particular value of $n > 0$, $m = (-1)^n$

Let $P(n)$ be the statement that after executing the program for a particular value of $n > 0$, $m = (-1)^n$. We will show that $P(n)$ is true for all $n > 0$ by induction.

Basis Step: $P(1)$ is true since $m = -1 = (-1)^1$.

Inductive Step: Suppose that $P(k)$ is true for some $k > 0$. We will show that $P(k + 1)$ is true.

After executing the program for $k + 1$, $m = -1 \cdot (-1)^k = (-1)^{k+1}$.

By the principle of mathematical induction, we conclude that $P(n)$ is true for all $n > 0$. ■

Question 14

Consider the following program:

```
procedure sum(n):
  s := 1
  i := 1
  while i<n:
    i := i + 1
    s := s + i
```

Show that after executing the program for a particular value of $n > 0$, $s = \frac{n(n+1)}{2}$

Let $P(n)$ be the statement that after executing the program for a particular value of $n > 0$, $s = \frac{n(n+1)}{2}$. We will show that $P(n)$ is true for all $n > 0$ by induction.

Basis Step: $P(1)$ is true since $s = 1 = \frac{1(1+1)}{2}$.

Inductive Step: Suppose that $P(k)$ is true for some $k > 0$. We will show that $P(k + 1)$ is true.

After executing the program for $k + 1$, $s = \frac{k(k+1)}{2} + (k + 1) = \frac{k(k+1)+2(k+1)}{2} = \frac{(k+1)(k+2)}{2}$.

By the principle of mathematical induction, we conclude that $P(n)$ is true for all $n > 0$. ■

Question 15

Consider the following program:

```
procedure iterative_fibonacci(n)
x := 0
y := 1
i := 1
while i < n
i := i + 1
z := x + y
x := y
y := z
```

Show that after executing the program for a particular value of $n > 0$, $y = f_n$, the n-th element of Fibonacci sequence.

Let $P(n)$ be the statement that after executing the program for a particular value of $n > 0$, $y = f_n$. We will show that $P(n)$ is true for all $n > 0$ by induction.

Basis Step:

1. $P(1)$ is true since $y = 1 = f_1$.
2. $P(2)$ is true since $y = 1 = f_2$.

Inductive Step: Suppose that $P(k)$ and $P(k+1)$ are true for some $k > 0$. We will show that $P(k+2)$ is true.

Assume $P(k)$ is true for some $k > 1$ and $P(k+1)$ is true.

Now consider the $(k+1)$-th generation

Before the iteration: $x = f_{k-1}$ and $y = f_k$, $i = k$.

During the iteration: $z = f_{k-1} + f_k = f_{k+1}$, $x = f_k$, $y = f_{k+1}$, $i = k + 1$.

After the iteration: $x = f_k$ and $y = f_{k+1}$.

Therefore, $P(k+2)$ is true.

By the principle of mathematical induction, we conclude that $P(n)$ is true for all $n > 0$. ■